DO NOW — RECALL

# Recursion: base cases and returns

Name: _____ Class: _____ Date: _____

**1. A C# function is declared int Double(int n). What does return n * 2; do when n is 4?**

**2. Starting total = 1, repeat total = total * i for i = 1, 2, 3. What is total after each iteration?**

**3. Values A, B and C are pushed onto a stack in that order. Which leaves first when popping?**

**4. A function AddTwo(x) returns x + 2. Calculate AddTwo(AddTwo(3)) and show the inner and outer calls.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

It sends the value 8 back to the caller. A return statement also ends the current invocation.

### 2. Model answer

1, 2 and 6. Each iteration multiplies the previous total by the new i.

### 3. Model answer

C. A stack is last in, first out (LIFO).

### 4. Model answer

The inner AddTwo(3) returns 5; then the outer AddTwo(5) returns 7.

### Vocabulary — know these terms

recursion — A function calling itself to solve a smaller problem.

base case — A condition that stops further recursive calls.

recursive case — A call to the same function with a smaller or simpler input.

call stack — Memory tracking active function calls and their local state.

return value — The result passed back to a caller.

NEW LEARNING

# Read, annotate and explain

### Start with a small input

Factorial(4) calls Factorial(3); it cannot finish multiplying until the inner call returns.

### Reach the stopping rule

Factorial(0) returns 1. It makes no further recursive call.

### Return through the calls

Factorial(1) = 1 × 1; Factorial(2) = 2 × 1; Factorial(3) = 3 × 2; Factorial(4) = 4 × 6 = 24.

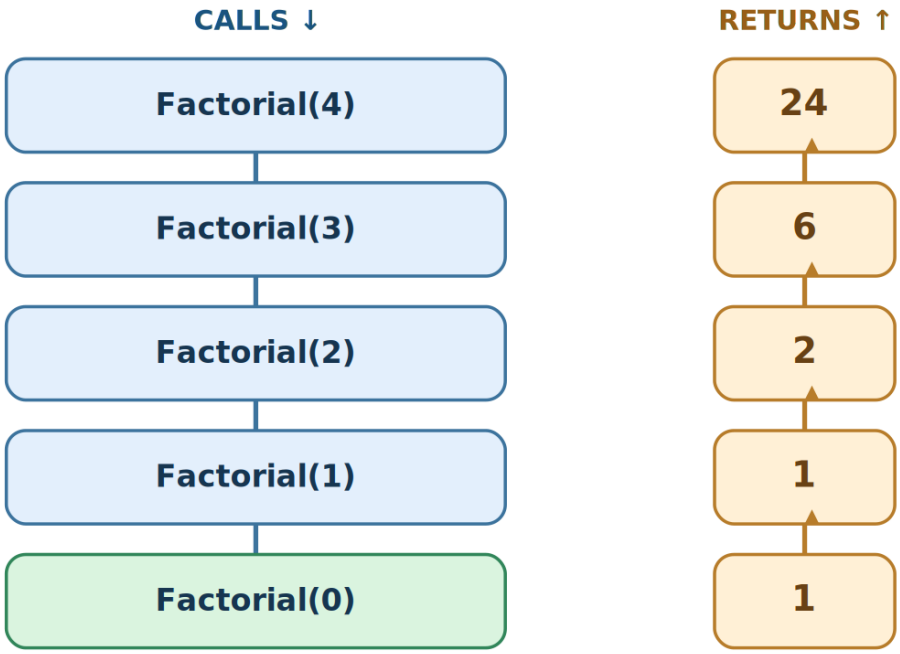
**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### What is the program actually doing?

Think: Why does the input get smaller? What stops another call?




Recursive calls moving downward and return values moving upwards. Factorial(4), Factorial(3), Factorial(2), Factorial(1), Factorial(0).

I DO — WORKED EXAMPLE

# Follow the worked method

### Trace Factorial(3). Show each call and its return value.

### Calls

3 → 2 → 1 → 0. The argument decreases by one each time.

### Base case

Factorial(0) returns 1. No more calls occur.

### Returns

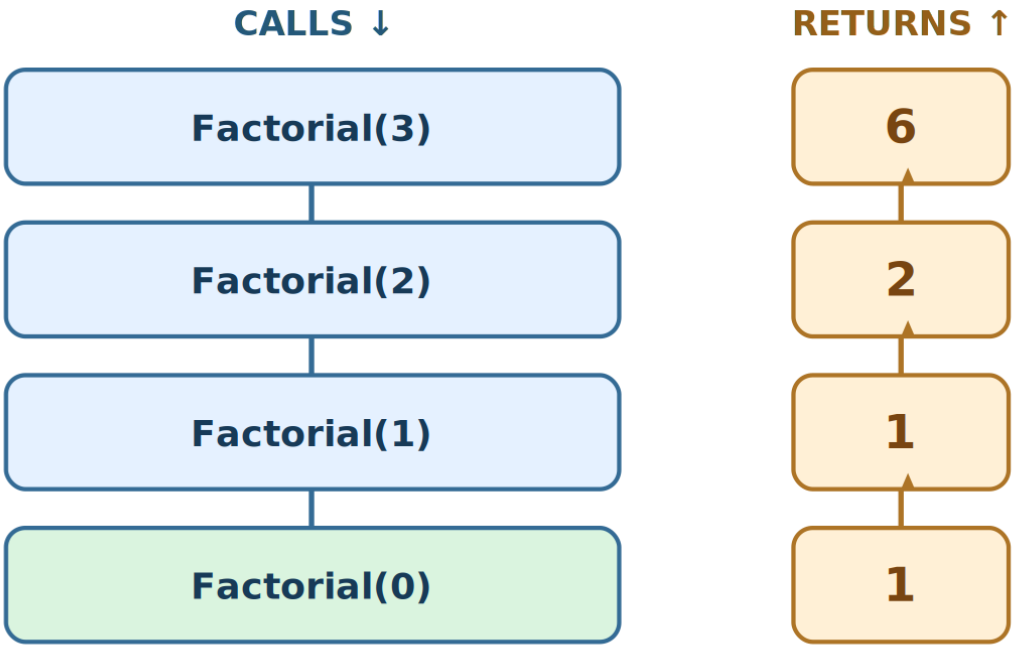
Factorial(1)=1, Factorial(2)=2, Factorial(3)=6. Final answer: 6.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Trace Factorial(3). Show each call and its return value.**

3 → 2 → 1 → 0. The argument decreases by one each time.




Call trace and return trace: Factorial(3); Factorial(2); Factorial(1); Factorial(0). Return values: 6, 2, 1, 1.

WE DO — GUIDED PRACTICE

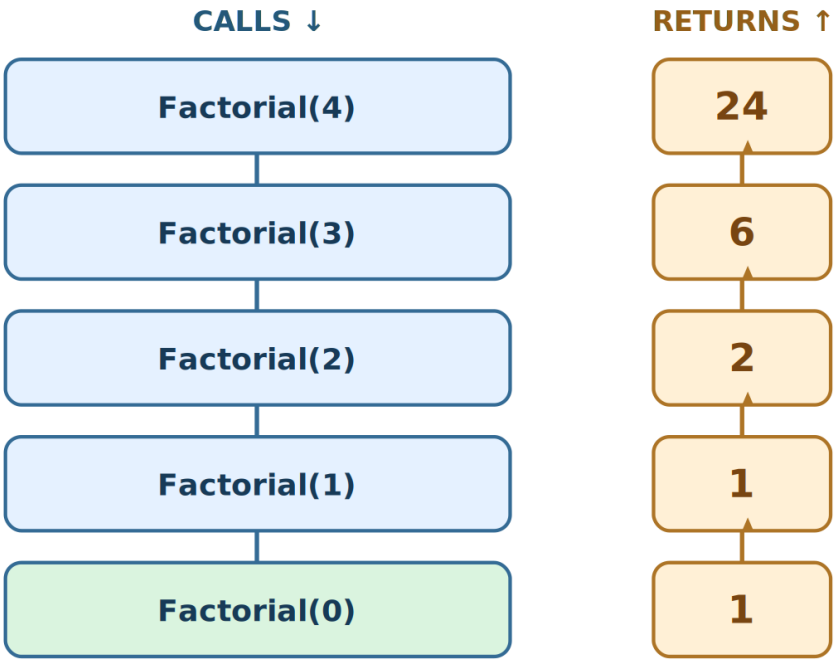
# Try → Discuss → Improve

# We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Trace Factorial(4) without running the program. Identify the base case and final returned value.**

Factorial(4) → ... → Factorial(0)




Call trace and return trace: Factorial(4); Factorial(3); Factorial(2); Factorial(1); Factorial(0). Return values: 24, 6, 2, 1, 1.

# WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

### Guided 1: Predict the next call 4 calls 3; 3 calls 2; 2 calls 1; 1 calls 0.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

---

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Complete the calls and returns for Factorial(2).

### Question 2

What happens if Factorial has no base case? Explain using the call stack.

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

Modify Factorial to reject negative input; write the C# guard and explain the reason.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**If a recursive call reaches its base case, all previous calls have already returned.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

# WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

### Guided 1 — original CHECK

Predict the first return

Factorial(0) returns 1.

Complete the return chain

Factorial(1)=1, Factorial(2)=2, Factorial(3)=6, Factorial(4)=24.

IMPROVE: Explain one correction to your own reasoning.